HongKong Metropolitan University

COMP 8090SEF
Data Structures And Algorithms

Self-study Report

Submission date: 9 April 2026

I declare that:

(i) I have read and checked that all parts of the project (including proposal, code programs, and reports), they are contributed by me, here submitted is original except for source material explicitly acknowledged;

(ii) the project, in parts or in whole, has not been submitted for more than one purpose without declaration;

(iii) I am aware of the University's policy and regulations on honesty in academic work and understand the possible consequence when breaching such policy and regulations;

(iv) I confirm that I have declared in the report about the usage of AI and other generative models, including but not limited to ChatGPT, LLaMA, Gemini, Mistral, and Stable Diffusion, and complied with the instructions provided by HKMU; and

(v) I am aware that I should be held responsible and liable to disciplinary actions, irrespective of whether I have signed the declaration and whether I have contributed, directly or indirectly, to the problematic contents.

I confirm that I have read through and understood the project requirements. I understand that failure to comply with the project requirements will result in score deduction.

NAME	SID
HE Xue	13927408

Self-study Report
Graph data structure and Cocktail Sort algorithm

1. Introduction

For Task 2 of the COMP8090SEF course study project, I selected the Graph data structure and the Cocktail Sort algorithm as my self-study topics.

Graphs are a versatile data structure that represents entities and their connections, much like a map of cities and roads. They are easy to conceptualize yet powerful for complex problems.

Cocktail sort, also known as shaker sort or bidirectional bubble sort, improves upon bubble sort by sorting in both directions, reducing passes and making it more efficient for nearly sorted data.

The report will introduce the abstract data type (ADT) for graphs, detail its main methods and applications, analyze the time complexity of cocktail sort with examples, and cover their development through Python implementations. This self-study has enhanced my knowledge of non-linear data structures and sorting optimizations.

The repository link is:
https://github.com/Xuewaonline/HEXue_SID13927408_COMP8090SEF_IndividualProject/tree/main/Task2_SelfStudy

The video link is:
https://www.youtube.com/watch?v=1EdxCD77W2k&t=30s

2. Data Structure: Graph

A graph is an abstract data type (ADT) that represents a set of objects called vertices and the relationships between them called edges. The ADT for a graph includes operations to add vertices and edges, remove them, and check for connections. In simple terms, the graph ADT is like a map of points and lines, where points are vertices and lines are edges. Vertices can be anything, like people or places, and edges show how they are linked, like friendships or roads.
(Appendix & References: Figure 0 Comparison Table of Graph)

2.1 Undirected Graph

An undirected graph is a graph where edges have no direction. This means if there is an edge between node A and node B, you can traverse it in both directions (from A to B and B to A). Edges represent symmetric relationships, like mutual friendships. There are no arrows on edges; they are just lines connecting nodes.
(Appendix & References: Figure 1 Data Structure (Undirected Graph))

Key Characteristics:

- Edges are bidirectional.
- No self-loops or multiple edges between the same pair unless specified as a multigraph.
- Represented using adjacency lists or adjacency matrices

Possible Applications: Road maps, friendship networks, or undirected connections in

computer networks.

Comments: Undirected Graph is simple for symmetric data, but it can't model one-way relationships.

2.2 Directed Graph

A directed graph is a graph where edges have a direction, indicated by arrows. An edge from A to B means you can go from A to B, but not necessarily B to A unless there's a separate edge. Edges represent asymmetric relationships, like one-way streets.
(Appendix & References: Figure 2 Data Structure (Directed Graph))

Key Characteristics:

- Edges are unidirectional (arrows point from source to destination).
- Can have cycles (loops back to start) .

Possible Applications: Web links (hyperlinks are one-way), task scheduling, or traffic flow with one-way streets.

Comments: Models real-world asymmetries; but more complex to traverse.

2.3 Weighted Graph

A weighted graph is a graph (can be undirected or directed) where each edge has a weight or cost associated with it, representing a value like distance, time, or price. Weights are numbers (integers or floats) on edges.
(Appendix & References: Figure 3 Data Structure (Weighted Graph))

Key Characteristics:

- Builds on undirected or directed graphs but adds numerical values to edges.
- No weights mean it's unweighted (all edges equal, like weight=1).

Possible Applications: GPS navigation (shortest path with distances), network routing (bandwidth as weights), or social graphs with friendship strength.

Comments: Handles real costs; but algorithms like shortest path are needed, adding complexity.

3. Algorithm: Cocktail Sort

Cocktail sort is a sorting algorithm that is a variation of bubble sort. It sorts a list by repeatedly going through it from left to right and then from right to left, swapping adjacent items if they are in the wrong order. The definition is that it bubbles the largest items to the end in one direction and the smallest to the beginning in the other direction. The name "cocktail" comes from the way it mixes or shakes the elements back and forth, similar to shaking a cocktail shaker.
(Appendix & References: Figure 4 Algorithm (Cocktail Sort))

Key Characteristics:

- Bidirectional Passes: Unlike Bubble Sort, which only passes in one direction, Cocktail Sort alternates between forward and backward passes. In the forward pass, adjacent elements are compared and swapped if out of order, moving larger elements to the right. In the backward pass, smaller elements are moved to the

left.

- Optimization Flag: It uses a flag to track if any swaps occurred in a pass. If no swaps happen in a full bidirectional pass, the array is sorted, allowing early termination.
- Time Complexity:

Worst-case: $O(n^2)$ – when the array is reverse-sorted, requiring many comparisons and swaps.
Average-case: $O(n^2)$ – but performs better than Bubble Sort due to bidirectional movement.
Best-case: $O(n)$ – for already sorted arrays, as it detects no swaps quickly.

Possible Applications:

- Data Processing: In software where data arrives incrementally and needs quick re-sorting, such as updating leaderboards in games or priority lists in task managers.
- File Sorting Utilities:Sorting short text files or logs.
- Educational Purposes: Used for test result sorting.

Comments:

Cocktail Sort is a straightforward improvement over Bubble Sort. Its main strength is early termination in best/average cases, but it remains inefficient for large, random datasets compared to divide-and-conquer sorts. One limitation is its quadratic time, which can be mitigated by better gap sequences (similar to Shell Sort). It's stable and in-place, which is advantageous for memory-constrained environments.

Code Demonstration:

```python
def cocktail_sort(arr):
    n = len(arr)  # Get array length
    swapped = True  # Flag to track if swaps occurred
    start = 0  # Start of unsorted portion
    end = n - 1  # End of unsorted portion
    while swapped:  # Continue until no swaps
        swapped = False
        # Forward pass: bubble largest to end
        for i in range(start, end):
            if arr[i] > arr[i + 1]:  # If out of order
                arr[i], arr[i + 1] = arr[i + 1], arr[i]  # Swap
                swapped = True
        if not swapped:  # If sorted, break
            break
        end -= 1  # Shrink end (largest fixed)
        swapped = False
        # Backward pass: bubble smallest to start
        for i in range(end - 1, start - 1, -1):
            if arr[i] > arr[i + 1]:  # If out of order
                arr[i], arr[i + 1] = arr[i + 1], arr[i]  # Swap
                swapped = True
        start += 1  # Shrink start (smallest fixed)
    return arr
```

Explanation of code:

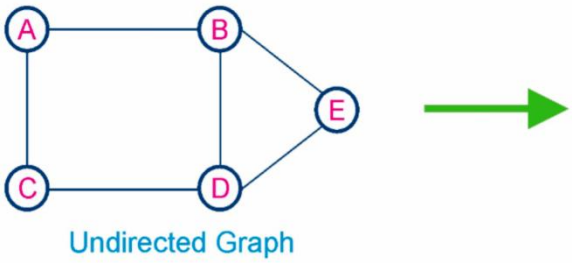
The function takes an array arr. It uses a while loop to repeat passes until no swaps. The forward loop (range(start, end)) swaps larger elements right. After, shrink end. The backward loop (range(end-1, start-1, -1)) swaps smaller left. Shrink start. The flag optimizes by exiting early if sorted. This demonstrates bidirectional logic simply. Test with [8,7,4,8,1]: After calls, returns [1,4,7,8,8].

Appendix & References:

Figure 0: Comparison Table of Graph

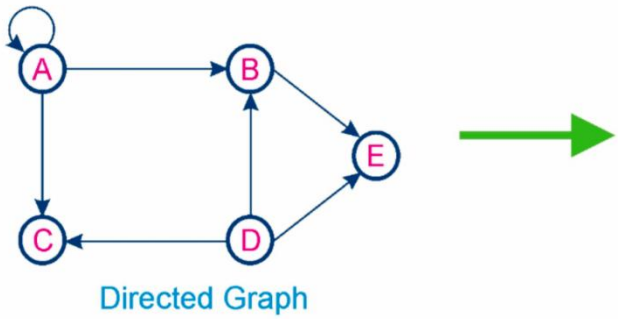
Aspect	Undirected Graph	Directed Graph	Weighted Graph
1. Direction	No direction (bidirectional)	Has direction (unidirectional)	Can be either, with weights on edges
2. Edge Representation	Simple line (A—B)	Arrow (A→B)	Line/arrow with number (A—50—B)
3. Example Relationship	Mutual friendship	X(Twitter)	Distance between cities
4. Traversal	Symmetric (both ways)	Follow arrows only	Consider weights for costs
5. Common Use	Social networks	Web links, workflows	Maps
6. Complexity	Simple	Handles asymmetries	Adds numerical analysis

Figure 1: Data Structure (Undirected Graph)



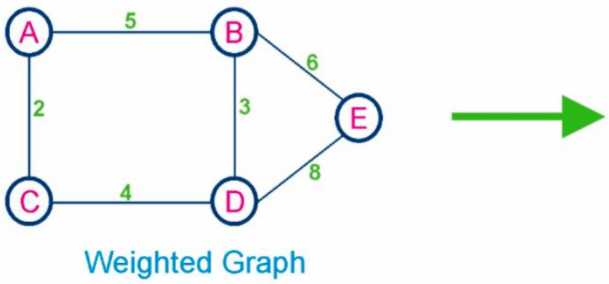
	A	B	C	D	E
A	0	1	1	0	0
B	1	0	0	1	1
C	1	0	0	1	0
D	0	1	1	0	1
E	0	1	0	1	0

Figure 2: Data Structure (Directed Graph)



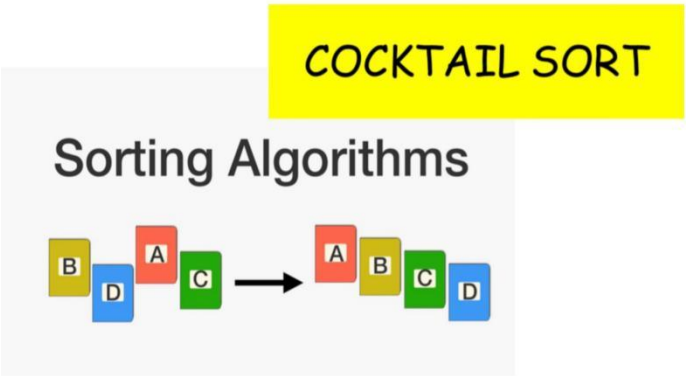
	A	B	C	D	E
A	1	1	1	0	0
B	0	0	0	0	1
C	0	0	0	0	0
D	0	1	1	0	1
E	0	0	0	0	0

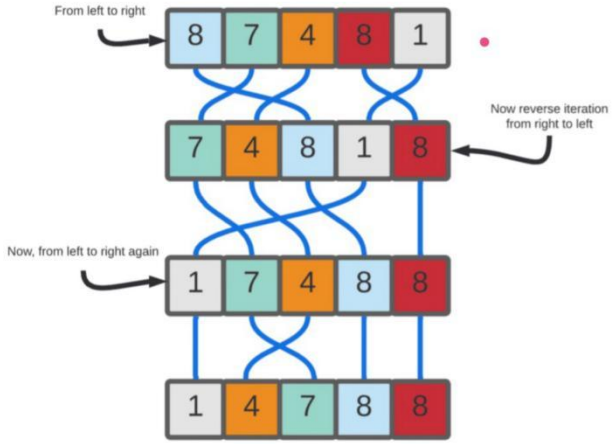
Figure 3: Data Structure (Weighted Graph)



	A	B	C	D	E
A	0	5	2	0	0
B	5	0	0	3	6
C	2	0	0	4	0
D	0	3	4	0	8
E	0	6	0	8	0

Figure 4: Algorithm (Cocktail Sort)





References:
Graph Representation
https://studyglance.in/ds/display.php?tno=33&topic=Graph-Representation
Cocktail Sort Steps
https://www.educba.com/cocktail-sort/